OurMocks

2027 CYCLE

MOCK · PRACTICE · CRACK

CUET UG

Computer Science

Sorting Algorithms & Complexity Analysis

Elementary to Advanced Sorting, Recurrence Relations, Invariants & Lower Bound Theory

INSIDE
Concept Notes

INSIDE
Solved Examples

INSIDE
Practice MCQs + Solutions

Aligned to the latest official syllabus · Concept-first explanations

www.ourmocks.com

Syllabus Alignment

Section / Unit	Official NTA Syllabus Pointers	Core Focus & High-Yield Exam Traps
Section B1: Topic 3 (Class 12)	Sorting Techniques: Bubble Sort, Selection Sort, Insertion Sort, dry runs, pass tracing, and swap count calculations.	Algorithmic invariants after pass k, best vs worst comparison counts, adaptive behavior with early-exit boolean flags, stability guarantees.
Section B1: Topic 3 (Class 12)	Divide & Conquer Sorting: Merge Sort, Quick Sort, partitioning strategies, recurrence trees, and auxiliary memory bounds.	Recurrence relations ($T(n) = 2T(\frac{n}{2}) + \Theta(n)$), stack memory requirements, Lomuto vs Hoare partitions, worst-case degradation on sorted inputs.
Section B1: Topic 3 (Class 12)	Complexity Theory & Python Implementation: Asymptotic time and space complexity, stability proofs, comparison lower bound, and Timsort.	Comparison lower bound $\Omega(n \log n)$ decision tree, difference between `list.sort()` and `sorted()`, adaptive properties of hybrid Timsort.

EXAM PATTERN

CUET Examination Directives: Questions on sorting algorithms frequently evaluate pass-by-pass array state tracing, total comparison counts versus total swap counts, algorithmic invariants, and worst-case triggers. Be prepared for direct questions identifying the algorithm from an intermediate pass state or calculating comparisons in Selection Sort (which **always** requires $\frac{n(n-1)}{2}$ comparisons regardless of input ordering).

Taxonomy & Foundations of Sorting Algorithms

Sorting is the algorithmic process of rearranging an arbitrary collection of items into a monotonic sequence defined by a strict total ordering relation (commonly non-decreasing order: $A[0] \leq A[1] \leq ... \leq A[n-1]$). In computer science and competitive examinations like CUET, sorting algorithms are systematically classified according to multiple formal dimensions:

Formal Classification Dimensions

1. Comparison vs Non-Comparison Sorts:

- **Comparison-based Sorts:** Formulate ordering decisions solely by evaluating pairs of keys using comparison operators ($<, >, \leq$). Algorithms such as Bubble, Selection, Insertion, Merge, and Quick Sort belong to this family. They are governed by the information-theoretic lower bound $\Omega(n \log n)$.
- **Non-Comparison Sorts:** Exploit internal data properties (e.g., integer keys or fixed-radix representations) without pairwise comparisons, such as Counting Sort, Radix Sort, and Bucket Sort. They achieve linear time $O(n+k)$ under constrained domains.

2. Space Efficiency & In-Place Property:

- **In-Place Sorts:** Require only $O(1)$ additional auxiliary memory beyond the input container (or $O(\log n)$ recursion stack space). Examples include Bubble Sort, Selection Sort, Insertion Sort, and Quick Sort.
- **Out-of-Place Sorts:** Require significant auxiliary buffers proportional to the input size, specifically $O(n)$ extra memory. The standard formulation of Merge Sort is an out-of-place algorithm.

Algorithmic Stability: Formal Definition & Significance

A sorting algorithm is formally designated as **stable** if and only if two items with identical keys appear in the output sequence in the exact same relative order as they occupied in the original input collection:

$$\forall i, j \in \{0, ..., n-1\}, \quad i < j \text{ and } \text{key}(A[i]) = \text{key}(A[j]) \Rightarrow \pi(i) < \pi(j)$$

where $\pi(k)$ denotes the final index of item $A[k]$ in the sorted array.

Algorithm	Stable?	In-Place?	Primary Reason for Stability / Instability
Bubble Sort	Yes	Yes	Adjacent swaps occur strictly when $A[j] > A[j+1]$; equal keys never swap.
Selection Sort	No	Yes	Long-distance swaps can propel an element past an identical counterpart.
Insertion Sort	Yes	Yes	Elements shift strictly while $A[j] > \text{key}$; stops immediately on equal key.
Merge Sort	Yes	Yes (aux $O(n)$)	Subarray merge takes left element when $L[i] \leq R[j]$, preserving order.
Quick Sort	No	Yes (aux $O(\log n)$)	Partitioning swaps elements across the pivot, violating original relative order.
Timsort (Python)	Yes	No (aux $O(n)$)	Engineered as a stable hybrid of Merge Sort and binary Insertion Sort.

WHY STABILITY MATTERS IN PRACTICAL MULTI-KEY SORTING

Consider a student database initially sorted alphabetically by Name. If we subsequently sort the list by Grade using a **stable** algorithm, students sharing the identical Grade will remain sorted alphabetically by Name. An unstable sort destroys the previous alphabetical order among tied grades.

Bubble Sort: Mechanics, Optimization & Invariants

Fundamental Operational Mechanics

Bubble Sort operates through repeated sequential passes over an array of size n. In each pass, adjacent elements $(A[j], A[j+1])$ are compared. If they violate the sorting criterion (i.e., $A[j] > A[j+1]$ for ascending order), they are swapped. Through this mechanism, the largest unsorted element in each pass "bubbles up" to its correct global terminal position.

```
def bubble_sort(arr):
    n = len(arr)
```

```
    for i in range(n - 1):          # Passes: 0 to n - 2
        for j in range(n - 1 - i):  # Comparisons shrink each pass
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
```

The Algorithmic Invariant of Bubble Sort

After the completion of pass k (where $k \in \{1, 2, ..., n-1\}$):

1. The k largest elements of the array reside in their final globally sorted positions within the rightmost subarray $A[n-k...n-1]$.
2. The remaining unsorted elements occupy indices 0 through $n-k-1$.
3. The total number of required passes is at most $n-1$.

Mathematical Complexity Analysis

- **Comparisons in Standard Bubble Sort:** Regardless of the initial array ordering, the nested loops execute a deterministic count of pairwise comparisons:

$$C(n) = \sum_{i=0}^{n-2}(n-1-i) = (n-1) + (n-2) + ... + 1 = \frac{n(n-1)}{2} = \Theta(n^2)$$

- **Swaps in Standard Bubble Sort:** Worst case (reversed array): $\frac{n(n-1)}{2} = \Theta(n^2)$ swaps. Best case (already sorted): 0 swaps.

Early-Exit Optimization (Adaptive Bubble Sort)

If an entire pass completes without a single swap, every adjacent pair satisfies $A[j] \leq A[j+1]$, proving the entire array is sorted. Introducing a boolean flag `swapped` reduces best-case time to $O(n)$ with $n-1$ comparisons and 0 swaps.

```
def adaptive_bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
```

OurMocks

PASS-BY-PASS TRACING OF BUBBLE SORT

Problem: Trace $A = [64, 34, 25, 12, 22]$ through Pass 1 and Pass 2 of standard Bubble Sort.
Solution:

- **Pass 1:** Compare $(64, 34) \rightarrow [34, 64, 25, 12, 22]$; $(64, 25) \rightarrow [34, 25, 64, 12, 22]$; $(64, 12) \rightarrow [34, 25, 12, 64, 22]$; $(64, 22) \rightarrow [34, 25, 12, 22, 64]$. Largest element (64) locked at index 4.
- **Pass 2:** Compare $(34, 25) \rightarrow [25, 34, 12, 22, 64]$; $(34, 12) \rightarrow [25, 12, 34, 22, 64]$; $(34, 22) \rightarrow [25, 12, 22, 34, 64]$. Second largest (34) locked at index 3.

Comparative Asymptotic Matrix & Selection Sort

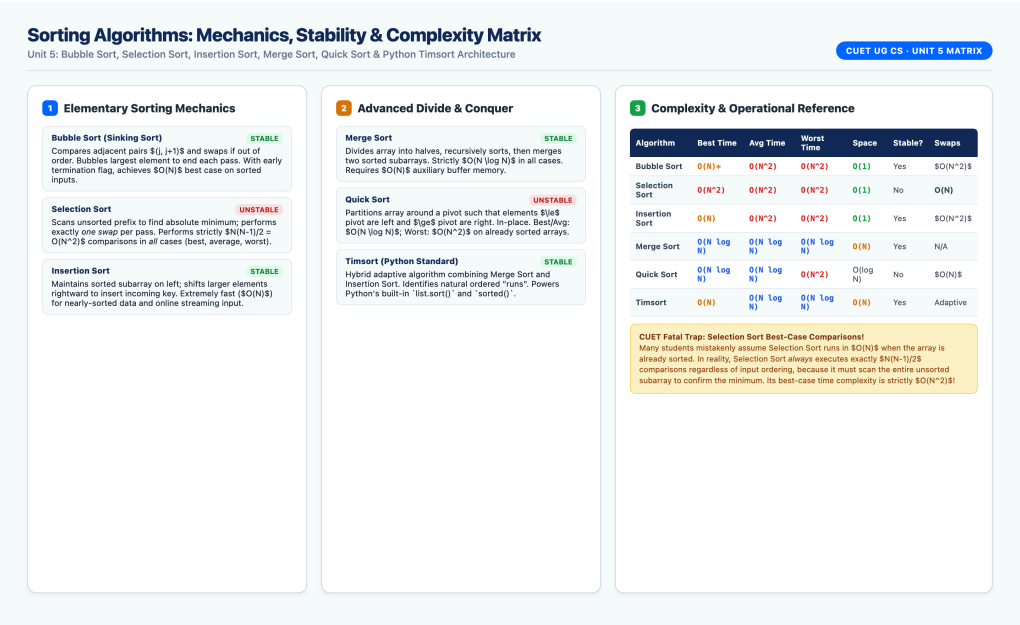


Figure 1: Comparative Asymptotic Matrix & Algorithmic Attributes across Standard Sorting Paradigms

Selection Sort: Mechanics & Philosophy

Selection Sort divides the array into a sorted left prefix and an unsorted right suffix. In each pass i (from 0 to $n-2$), it linear-scans the unsorted subarray $A[i...n-1]$ to locate the index of the minimum element, and executes a single swap placing that minimum at index i.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Key Properties & Fatal CUET Distinctions

1. **Minimal Data Movement ($O(n)$ Swaps):** Selection Sort performs at most $n-1$ swaps across its entire execution. When write operations to flash memory or EEPROM are physically costly, Selection Sort minimizes memory wear compared to Bubble Sort's $O(n^2)$ swaps.
2. **Non-Adaptive Comparisons:** Regardless of whether the input is sorted, reversed, or random, the inner loop **must** check every remaining unsorted position to confirm the absolute minimum. Total comparisons are strictly $\frac{n(n-1)}{2}$ in ALL cases (Best, Average, and Worst).
3. **Instability:** A swap over a long distance can leap over identical keys, breaking stability.

Deep-Dive: Algorithmic Invariant of Selection Sort

After completing pass k (where $k \in \{1, 2, ..., n-1\}$):

1. The k smallest elements of the entire array occupy the first k positions $A[0...k-1]$ in their globally final sorted order.
2. Every element in $A[0...k-1]$ is less than or equal to every element in the unsorted suffix $A[k...n-1]$.
3. Unlike Bubble Sort (which fixes elements from right to left), Selection Sort fixes elements strictly from **left to right**.

Mathematical Proof of Instability

Consider $A = [(4_a, \text{apple}), (4_b, \text{banana}), (2, \text{cherry})]$.

- **Pass 1:** Minimum is $(2, \text{cherry})$ at index 2. Swap $A[0]$ and $A[2]$:

$$A \to [(2, \text{cherry}), (4_b, \text{banana}), (4_a, \text{apple})]$$

- **Pass 2:** Subarray is $[(4_b, \text{banana}), (4_a, \text{apple})]$. Minimum is 4_b. No swap.
- **Result:** 4_b now precedes 4_a, proving Selection Sort is inherently **unstable**.

SELECTION SORT DRY RUN & SWAP COUNTS

Problem: Given $A = [29, 10, 14, 37, 13]$, trace each pass of Selection Sort and determine the total comparisons and swaps executed.
Solution:

- **Pass 1 ($i = 0$):** Scan $A[0...4]$. Minimum is 10 at index 1. Swap $A[0], A[1] \to [10, 29, 14, 37, 13]$ (1 swap, 4 comparisons).
- **Pass 2 ($i = 1$):** Scan $A[1...4]$. Minimum is 13 at index 4. Swap $A[1], A[4] \to [10, 13, 14, 37, 29]$ (1 swap, 3 comparisons).
- **Pass 3 ($i = 2$):** Scan $A[2...4]$. Minimum is 14 at index 2 (`min_idx == i`). No swap $\to [10, 13, 14, 37, 29]$ (0 swaps, 2 comparisons).
- **Pass 4 ($i = 3$):** Scan $A[3...4]$. Minimum is 29 at index 4. Swap $A[3], A[4] \to [10, 13, 14, 29, 37]$ (1 swap, 1 comparison).
- **Total Comparisons:** $4 + 3 + 2 + 1 = 10 = \frac{5 \times 4}{2}$.
- **Total Swaps:** $1 + 1 + 0 + 1 = 3$ swaps ($\leq n - 1 = 4$).

Insertion Sort: Shifting Mechanics & Inversion Theory

Algorithmic Philosophy & Mechanism

Insertion Sort conceptually mirrors how players arrange playing cards. The array is partitioned into a sorted prefix and an unsorted suffix. Iterating from index 1 to $n - 1$, the algorithm extracts the current element as a `key` and iteratively shifts larger elements of the sorted prefix one position to the right until the correct insertion slot is cleared:

```
def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
```

OurMocks

The Critical Invariant of Insertion Sort

After pass k (where $k \in \{1, 2, ..., n - 1\}$, processing element at index k):

1. The prefix subarray $A[0...k]$ consists of elements originally in $A[0...k]$, now rearranged in **mutually sorted order**.
2. **CRITICAL CUET TRAP:** Elements in $A[0...k]$ are **NOT** necessarily in their globally final positions! A smaller element encountered in a future pass will cause elements in this prefix to shift further right.

Inversion Counting & Adaptive Linear Run-Time

An **inversion** in an array A is a pair (i, j) such that $i < j$ and $A[i] > A[j]$.

- In an already sorted array, inversions $= 0$.
- In a reverse sorted array, inversions $= \frac{n(n-1)}{2}$.
- Each shift in Insertion Sort eliminates exactly one inversion. Thus:

$$\text{Total Shifting Operations} = \text{Total Number of Inversions in } A$$

BEST-CASE EFFICIENCY OF INSERTION SORT

When an array is already sorted, the condition `arr[j] > key` fails on the very first comparison ($j = i - 1$). The `while` loop never executes its body, resulting in exactly 1 comparison per element and 0 shifts. Total time is strictly $\Theta(n)$. This makes Insertion Sort the premier algorithm for **nearly sorted** or streaming data!

PASS TRACING OF INSERTION SORT

Problem: Trace Insertion Sort on $A = [9, 5, 1, 4, 3]$ for the first 3 passes.
Solution:

- **Pass 1 ($i = 1$, key $= 5$):** Compare with 9 ($9 > 5$). Shift 9 right. Insert 5 at index 0 $\rightarrow [5, 9, 1, 4, 3]$. Prefix $[5, 9]$ sorted.
- **Pass 2 ($i = 2$, key $= 1$):** Compare with 9 (shift), compare with 5 (shift). Insert 1 at index 0 $\rightarrow [1, 5, 9, 4, 3]$. Prefix $[1, 5, 9]$ sorted.
- **Pass 3 ($i = 3$, key $= 4$):** Compare with 9 (shift), compare with 5 (shift), compare with 1 ($1 < 4$, stop). Insert 4 at index 2 $\rightarrow [1, 4, 5, 9, 3]$. Prefix $[1, 4, 5, 9]$ sorted.

Sorting Pass Tracing & Partition Lifecycle

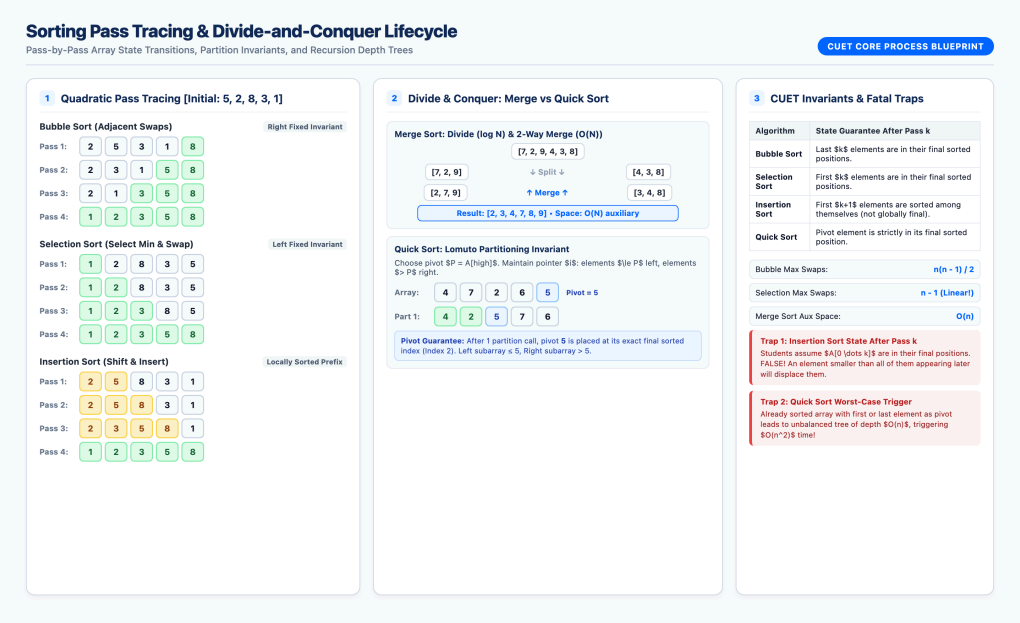


Figure 2: Pass Tracing Mechanics, Invariant Guarantees & Divide-and-Conquer Partitioning Lifecycles

Comprehensive Invariant Comparison Table

Algorithm	Subarray Invariant After Pass k	CUET Practical Identification Rule
Bubble Sort	Rightmost k elements $A[n-k \dots n-1]$ are in final global sorted positions.	Check right tail: the largest k items must appear in strictly ascending order at the end.
Selection Sort	Leftmost k elements $A[0 \dots k-1]$ are in final global sorted positions.	Check left prefix: the smallest k items appear in strictly ascending order at the start.
Insertion Sort	Left prefix $A[0 \dots k]$ is mutually sorted among itself; not globally final.	Check left prefix: elements $0 \dots k$ are sorted, but subsequent numbers may be smaller.
Quick Sort	Pivot element is strictly locked into its globally final sorted position.	All elements to pivot's left are $\leq$ pivot; all to its right are $\geq$ pivot.

Merge Sort: Divide-and-Conquer & Recurrence Relations

Algorithmic Philosophy

Merge Sort is a quintessential divide-and-conquer algorithm proposed by John von Neumann in 1945. It breaks a problem down into smaller independent subproblems, solves them recursively, and combines their solutions:

1. **Divide:** Find midpoint $m = \lfloor \frac{l+r}{2} \rfloor$ and split array $A[l \dots r]$ into two sub-arrays.
2. **Conquer:** Recursively sort left half $A[l \dots m]$ and right half $A[m+1 \dots r]$ until base case ($l \geq r$) is reached.

3. **Combine:** Merge two sorted halves into a unified sorted sequence using two pointers and an auxiliary buffer.

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:  # '<=' maintains stability!
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Recurrence Relation & Master Theorem Proof

Let $T(n)$ represent the worst-case runtime for an input array of length n:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n), \quad T(1) = \Theta(1)$$

Applying Case 2 of the Master Theorem ($a = 2, b = 2, f(n) = \Theta(n)$):

- Critical exponent: $\log_b(a) = \log_2(2) = 1$.
- Since $f(n) = \Theta\left(n^{\log_b a}\right) = \Theta(n^1)$, the solution is:

$$T(n) = \Theta\left(n^{\log_b a} \log n\right) = \Theta(n \log n)$$

Crucial Performance Attributes for CUET

- **Guaranteed Asymptotics:** Merge Sort guarantees $O(n \log n)$ runtime across **all** cases (Best, Average, and Worst).
- **Auxiliary Space Burden:** The merge operation requires a temporary buffer of size n to merge elements without overwriting unvisited entries. Its auxiliary space complexity is strictly $O(n)$.
- **Stability:** By using the comparison condition `left[i] <= right[j]`, any element from the left half is chosen first in case of a tie, guaranteeing stability.
- **Linked List Sorting:** Merge Sort is the algorithm of choice for sorting singly linked lists because sequential merge requires $O(1)$ extra space without memory reallocation.

MERGE SORT TREE & INVERSION COUNTING TRACE

Problem: For $A = [8, 4, 2, 1]$, trace the recursive divide-and-conquer tree and count the total inversions resolved during merge steps.
Solution:

- **Divide Level 1:** Split into $[8, 4]$ and $[2, 1]$.
- **Divide Level 2:** Split into singletons [8], [4], [2], [1].
- **Merge Level 1:**
  - Merge [8] and [4]: $8 > 4 \rightarrow 4$ placed first (1 inversion). Result: $[4, 8]$.
  - Merge [2] and [1]: $2 > 1 \rightarrow 1$ placed first (1 inversion). Result: $[1, 2]$.
- **Merge Level 2:**
  - Merge $[4, 8]$ and $[1, 2]$:
    - Compare 4 and 1: $4 > 1 \rightarrow$ place 1. Both remaining left elements (4 and 8) form inversions (2 inversions).
    - Compare 4 and 2: $4 > 2 \rightarrow$ place 2 (2 inversions).
    - Copy remaining left elements: $[4, 8]$.
- **Total Inversions:** $1 + 1 + 2 + 2 = 6 = \frac{4 \times 3}{2}$ (maximum for reverse-sorted array).

Quick Sort: Partitioning Schemes & Degeneracy Analysis

Mechanics: Pivot Selection & Partitioning

Quick Sort (invented by Tony Hoare in 1959) is an in-place divide-and-conquer sorting algorithm. Unlike Merge Sort, which does all work during the combine phase, Quick Sort executes its entire computational work during the **divide (partitioning)** phase.

```
def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)   # Sort left of pivot
        quick_sort(arr, pi + 1, high)  # Sort right of pivot

def partition(arr, low, high):
    pivot = arr[high]                  # Lomuto scheme: last element
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
```

Partitioning Schemes: Lomuto vs Hoare

- **Lomuto Partition:** Chooses the last element as pivot. Maintains a single pointer i marking the boundary of elements $\leq$ pivot. Executes more swaps on average ($\sim \frac{n}{2}$).
- **Hoare Partition:** Uses two pointers moving from opposite ends toward each other. Performs approximately three times fewer swaps than Lomuto and handles duplicates more efficiently.

Mathematical Analysis & The Degeneracy Catastrophe

Best & Average Case ($O(n \log n)$): When the pivot partitions the array into two roughly balanced halves (e.g., $\frac{n}{2}$ elements each):

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) = \Theta(n \log n)$$

The recursion tree achieves optimal depth $\log_2 n$. Stack space: $O(\log n)$.

Worst Case ($O(n^2)$): Occurs when the pivot is consistently the minimum or maximum element (e.g., already sorted array with first or last element chosen as pivot):

$$T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$$

Recursion depth degrades to n. Stack space: $O(n)$.

CUET HIGH-YIELD TRAP: QUICK SORT ON SORTED ARRAYS

If an array is **already sorted** in ascending order and the naive implementation selects `arr[0]` or `arr[n-1]` as the pivot, Quick Sort triggers its **worst-case** performance $O(n^2)$! To mitigate this, robust implementations utilize **Randomized Pivot Selection** or the **Median-of-Three** heuristic (`median(first, middle, last)`).

LOMUTO PARTITION STEP-BY-STEP EXECUTION TRACE

Problem: Trace the Lomuto partition algorithm on array $A = [25, 50, 15, 35, 20]$ with the last element (20) as pivot.
Solution:

- Initial setup: Pivot = 20, `low = 0`, `high = 4`, pointer `i = low - 1 = -1`.
- **Step 1 ($j = 0$, $A[0] = 25$):** Is $25 \leq 20$? No. `i` remains `-1`. Array: $[25, 50, 15, 35, 20]$.
- **Step 2 ($j = 1$, $A[1] = 50$):** Is $50 \leq 20$? No. `i` remains `-1`. Array: $[25, 50, 15, 35, 20]$.
- **Step 3 ($j = 2$, $A[2] = 15$):** Is $15 \leq 20$? Yes! Increment `i = 0`. Swap $A[i]$ and $A[j]$ (swap $A[0]$ and $A[2]$):
$$A \rightarrow [15, 50, 25, 35, 20]$$
.
- **Step 4 ($j = 3$, $A[3] = 35$):** Is $35 \leq 20$? No. `i` remains `0`.
- **Final Pivot Placement:** Swap $A[i+1]$ with $A[\text{high}]$ (swap $A[1]$ and $A[4]$):
$$A \rightarrow [15, 20, 25, 35, 50]$$
.
- **Result:** Pivot 20 is locked at index 1. Left subarray $[15] \leq 20$, Right subarray $[25, 35, 50] > 20$. Returns pivot index `1`.

Python Built-in Sorting & Comparison Lower Bounds

Python's Native Sorting Engine: Timsort

Both Python's `list.sort()` and built-in `sorted()` employ **Timsort**, an ultra-optimized hybrid adaptive sorting algorithm created in 2002 by Tim Peters for CPython.

Feature	Mechanisms & Architecture	CUET Examination Focus
Run Detection	Scans input for natural monotonically ascending or strictly descending runs. Descending runs are reversed in-place.	Exploits pre-existing partial order; achieves linear $O(n)$ time on already sorted data.
Minrun & Insertion	If natural runs are smaller than a threshold (`minrun`, typically 32 or 64), extends run via Binary Insertion Sort.	Binary Insertion Sort minimizes comparisons using binary search to find insertion slot.
Merge Stack	Runs are pushed onto a stack and merged following strict invariant rules ($A > B + C$ and $B > C$).	Ensures balanced 2-way merges; guarantees worst-case $O(n \log n)$ time and $O(n)$ space.
Galloping Mode	When one run consistently wins during merging, switches to exponential search (gallop).	Skips massive contiguous blocks of elements, drastically outperforming standard merge.

Built-in Functions: `list.sort()` vs `sorted()`

- `list.sort(*, key=None, reverse=False)`: Modifies the original list **in-place**. Returns `None`. Cannot be called on immutable tuples or strings.
- `sorted(iterable, *, key=None, reverse=False)`: Accepts **any** iterable (list, tuple, dict, set, string). Constructs and returns a **new list** containing sorted items. Original iterable remains completely untouched.

The Information-Theoretic Comparison Lower Bound

THEOREM: LOWER BOUND FOR COMPARISON-BASED SORTING

Any comparison-based sorting algorithm must perform at least $\Omega(n \log n)$ comparisons in the worst case to sort n distinct elements.

Proof via Decision Tree Model:

1. An array of n distinct elements has $n!$ possible permutations. Exactly one permutation represents the sorted sequence.
2. A comparison sort can be modeled as a binary decision tree where each internal node represents a comparison ($A[i] \leq A[j]$) and each leaf represents a possible permutation.
3. To correctly identify the sorted order among all permutations, the decision tree must have at least $L \geq n!$ leaves.
4. A binary tree of height h can have at most 2^h leaves. Therefore:
$$2^h \geq n! \Rightarrow h \geq \log_2(n!)$$
5. Applying Stirling's Approximation ($\ln(n!) \approx n \ln n - n$):
$$h \geq \log_2(n!) = \sum_{k=1}^{n} \log_2 k \geq \sum_{k=\lfloor \frac{n}{2} \rfloor}^{n} \log_2\left(\frac{n}{2}\right) = \frac{n}{2} \log_2\left(\frac{n}{2}\right) = \Omega(n \log n)$$
6. Since the height h corresponds to the maximum number of comparisons on the worst-case execution path, no comparison sort can beat $\Omega(n \log n)$.

DECISION TREE HEIGHT FOR SMALL ARRAYS

Problem: Using the decision tree model, calculate the minimum number of comparisons required in the worst case to sort:

1. An array of 3 distinct elements ($n = 3$).
2. An array of 5 distinct elements ($n = 5$).

Solution:

- **Case 1 ($n = 3$):** Total permutations $L = 3! = 6$. Minimum height $h \geq \lceil \log_2 6 \rceil = 3$. Hence, at least 3 comparisons are required in the worst case to sort 3 elements.

- **Case 2 ($n = 5$):** Total permutations $L = 5! = 120$. Since $2^6 = 64 < 120$ and $2^7 = 128 \geq 120$, minimum height $h \geq \lceil \log_2 120 \rceil = 7$. Hence, at least 7 comparisons are required in the worst case to sort 5 distinct elements.

OurMocks

Master Synthesis Mindmap: Sorting Algorithms



Figure 3: Master Synthesis Mindmap: Elementary vs Advanced Sorting, Invariants, Stability & Lower Bounds

OurMocks

Cheatsheet & The Seven Fatal CUET Traps

RAPID REVISION CHEATSHEET: SORTING ALGORITHMS & COMPLEXITIES

- **Bubble Sort:** Adjacent swaps. Pass k locks k largest items at right. Best: $O(n)$ (adaptive with flag), Worst: $O(n^2)$. Max comparisons: $\frac{n(n-1)}{2}$. Stable. In-place.
- **Selection Sort:** Select minimum in unsorted suffix, 1 swap per pass. Pass k locks k smallest items at left. Comparisons: **Always** $\frac{n(n-1)}{2}$ in ALL cases! Swaps: At most $n-1$. Unstable. In-place.
- **Insertion Sort:** Inserts key into sorted left prefix by shifting. Best: $O(n)$ (0 shifts), Worst: $O(n^2)$. Shifting steps = Inversion count. Stable. In-place. Excellent for small or nearly sorted arrays.
- **Merge Sort:** Divide and conquer $(2T(\frac{n}{2}) + \Theta(n))$. Best, Avg, Worst: $\Theta(n \log n)$. Auxiliary memory: $O(n)$ (Out-of-place). Stable.
- **Quick Sort:** Divide and conquer via pivot partitioning. Best & Avg: $\Theta(n \log n)$. Worst: $\Theta(n^2)$ (sorted input with naive pivot). Aux space: $O(\log n)$ stack. Unstable.
- **Timsort:** Hybrid of Merge Sort + Binary Insertion Sort. Used in Python. Best: $O(n)$, Avg & Worst: $O(n \log n)$. Space: $O(n)$. Stable.
- **Comparison Lower Bound:** $\Omega(n \log n)$ comparisons proven via binary decision tree with $n!$ permutation leaves.

THE SEVEN FATAL CUET TRAPS IN SORTING ALGORITHMS

1. **The Selection Sort Best-Case Trap:** Students assume Selection Sort runs in $O(n)$ time when the array is already sorted. **FALSE!** Selection Sort **always** executes $\frac{n(n-1)}{2}$ comparisons because the inner loop must scan every remaining element to prove it is the minimum. Best-case time is strictly $O(n^2)$.
2. **The Insertion Sort Invariant Trap:** Students mistakenly believe that after pass k, the elements $A[0...k]$ are in their globally final positions. **FALSE!** They are only sorted among themselves; a subsequent smaller number will displace them.
3. **The Quick Sort Sorted Input Trap:** When presented with an already sorted array, naive Quick Sort does **not** run in $O(n \log n)$; it degrades to its worst-case runtime $O(n^2)$ and stack depth $O(n)$.
4. **The `list.sort()` Return Value Trap:** Writing `res = my_list.sort()` assigns `None` to `res` because `list.sort()` modifies in-place and returns nothing. Use `res = sorted(my_list)` to obtain a new list.
5. **Stability Misconceptions:** Confusing Selection Sort with Insertion Sort. Selection Sort is **unstable** due to long-range swaps; Insertion Sort is **stable** because it shifts elements and halts immediately upon encountering an equal key.
6. **Number of Passes in Quadratic Sorts:** To sort an array of n elements using Bubble, Selection, or Insertion Sort, exactly $n-1$ passes are required in the worst case (not n).
7. **Merge Sort Space Complexity:** Believing Merge Sort is in-place. Standard Merge Sort strictly requires $O(n)$ auxiliary array buffer space to merge two sorted subarrays.

CUET Focused Multiple Choice Questions (Part 1)

Q1. An array containing n elements is already sorted in strictly ascending order. Which of the following sorting algorithms will still perform exactly $\frac{n(n-1)}{2}$ comparisons to verify the sorted order?

(A) Selection Sort
(B) Optimized Bubble Sort with swap flag
(C) Insertion Sort
(D) Timsort (Python default)

Sol. Q1. Selection Sort does not check whether elements are already in order; its inner loop iterates exhaustively through the remaining unsorted subarray in every pass to find the index of the minimum. Thus, it deterministically executes $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$ comparisons across all cases (Best, Average, Worst).

Answer: (A)

Q2. Consider the array $A = [42, 16, 84, 12, 77, 26]$. What will be the exact contents of the array after the completion of the **second pass** of standard Bubble Sort?

(A) $[16, 12, 26, 42, 77, 84]$
(B) $[16, 12, 42, 26, 77, 84]$
(C) $[12, 16, 26, 42, 77, 84]$
(D) $[16, 42, 12, 77, 26, 84]$

Sol. Q2. Let us trace Bubble Sort pass-by-pass:

- Pass 1: Compare adjacent pairs $(42, 16) \to [16, 42, ...]$; $(42, 84)$ no swap; $(84, 12) \to [..., 12, 84, ...]$; $(84, 77) \to [..., 77, 84, 26]$; $(84, 26) \to [16, 42, 12, 77, 26, 84]$.
- Pass 2: Compare $(16, 42)$ no swap; compare $(42, 12) \to$ swap $\to [16, 12, 42, 77, 26, 84]$; compare $(42, 77)$ no swap; compare $(77, 26) \to$ swap $\to [16, 12, 42, 26, 77, 84]$.

Answer: (B)

Q3. Which of the following statements correctly characterizes an algorithmic invariant of **Insertion Sort** after completing pass k (where elements from index 0 to k have been processed)?

(A) The k largest elements are fixed in their globally final sorted positions at the right end of the array.
(B) The k smallest elements are fixed in their globally final sorted positions at the left end of the array.
(C) The prefix subarray $A[0...k]$ consists of elements originally in $A[0...k]$ sorted among themselves, but not necessarily in their final global positions.
(D) The element originally at index k is guaranteed to be placed at its final global sorted position.

Sol. Q3. In Insertion Sort, after pass k, the subarray $A[0...k]$ is mutually sorted among itself. However, none of these elements are guaranteed to be in their globally final positions because any subsequent element in $A[k+1...n-1]$ that is smaller than elements in $A[0...k]$ will cause them to shift rightward in later passes.

Answer: (C)

Q4. What is the worst-case time complexity of Quick Sort, and under what specific input condition does this worst case typically occur when the last element is naively chosen as the pivot?

(A) $O(n \log n)$, when the array contains random elements
(B) $O(n \log n)$, when the array is already sorted in reverse order
(C) $O(n^2)$, when all elements in the array are distinct and in random order
(D) $O(n^2)$, when the input array is already sorted in ascending or descending order

Sol. Q4. When the input array is already sorted and the pivot is chosen as the last (or first) element, the partition produces the most unbalanced split possible: one subproblem of size $n-1$ and one of size 0. The recurrence becomes $T(n) = T(n-1) + \Theta(n)$, solving to $\Theta(n^2)$.

Answer: (D)

Q5. Why is standard Merge Sort classified as an **out-of-place** sorting algorithm while Bubble Sort and Selection Sort are classified as **in-place**?

(A) Merge Sort requires an auxiliary buffer of size $O(n)$ to merge two sorted subarrays without overwriting data, whereas Bubble and Selection Sort require only $O(1)$ extra space.
(B) Merge Sort cannot be implemented recursively, whereas Bubble Sort uses the system call stack.
(C) Merge Sort has a time complexity of $O(n \log n)$ while in-place algorithms are strictly restricted to $O(n^2)$.
(D) Merge Sort requires random access memory while Bubble Sort works on sequential tape drives.

Sol. Q5. An algorithm is in-place if it requires $O(1)$ auxiliary memory (beyond negligible loop variables or $O(\log n)$ recursion stack frames). Standard Merge Sort requires an auxiliary temporary array of size n to merge two sorted halves, giving it an auxiliary space complexity of $O(n)$.

Answer: (A)

CUET Focused Multiple Choice Questions (Part 2)

Q6. What is the output of the following Python code snippet?

```
nums = [15, 3, 9, 22, 1]
result = nums.sort()
print(result, nums)
```

(A) `[1, 3, 9, 15, 22] [1, 3, 9, 15, 22]` **(B)** `None [1, 3, 9, 15, 22]`
(C) `[1, 3, 9, 15, 22] [15, 3, 9, 22, 1]` **(D)** `None [15, 3, 9, 22, 1]`

Sol. Q6. In Python, `list.sort()` sorts the list in-place and returns `None`. Therefore, `result` evaluates to `None`, while the original list `nums` is modified to `[1, 3, 9, 15, 22]`. To create a new sorted list without modifying the original, one must use `result = sorted(nums)`.

Answer: (B)

Q7. Which of the following sorting algorithms is inherently **unstable**, and what causes its instability?

(A) Bubble Sort, because adjacent elements are swapped when out of order.
(B) Merge Sort, because elements are divided into two equal halves.
(C) Selection Sort, because long-range swaps can jump an element over an identical counterpart.
(D) Insertion Sort, because elements are shifted one position to the right.

Sol. Q7. Selection Sort is inherently unstable. When it identifies the minimum element in the unsorted suffix, it swaps it with the current position i. This long-distance swap can transport an element past an identical key located between i and the minimum, reversing their original relative order.

Answer: (C)

Q8. Based on the decision tree model of computation, what is the theoretical lower bound on the number of comparisons required in the worst case by any comparison-based sorting algorithm to sort n distinct elements?

(A) $\Omega(n)$ **(B)** $\Omega(\log n)$
(C) $\Omega(n^2)$ **(D)** $\Omega(n \log n)$

Sol. Q8. A decision tree sorting n distinct items must contain at least $n!$ leaves corresponding to all possible permutations. A binary tree of height h can have at most 2^h leaves. Hence $2^h \geq n!$, which implies $h \geq \log_2(n!) = \Omega(n \log n)$. Therefore, no comparison-based sorting algorithm can achieve a worst-case time faster than $\Omega(n \log n)$.

Answer: (D)

Q9. In an array of size n, what is the maximum number of element swaps executed by **Selection Sort** across its entire execution in the worst case?

(A) $n-1$ swaps **(B)** $\frac{n(n-1)}{2}$ swaps
(C) $n \log n$ swaps **(D)** $2n$ swaps

Sol. Q9. Selection Sort performs at most one swap per outer loop iteration. Since the outer loop executes for $i = 0, 1, ..., n-2$ (a total of $n-1$ passes), the absolute maximum number of swaps across the entire algorithm is $n-1$. This $O(n)$ swap bound makes Selection Sort ideal when write operations are physically expensive.

Answer: (A)

Q10. What recurrence relation represents the run-time complexity of standard Merge Sort, and what is its asymptotic solution by the Master Theorem?

(A) $T(n) = T(n-1) + \Theta(n) \to \Theta(n^2)$ **(B)** $T(n) = 2T(\frac{n}{2}) + \Theta(n) \to \Theta(n \log n)$
(C) $T(n) = 2T(\frac{n}{2}) + \Theta(1) \to \Theta(n)$ **(D)** $T(n) = T(\frac{n}{2}) + \Theta(1) \to \Theta(\log n)$

Sol. Q10. Merge Sort divides the array into two subproblems of size $\frac{n}{2}$ (costing $2T(\frac{n}{2})$) and merges them in linear time ($\Theta(n)$). By Master Theorem Case 2, where $a = 2, b = 2, f(n) = \Theta(n^{\log_2 2}) = \Theta(n)$, the solution is $T(n) = \Theta(n \log n)$.

Answer: (B)

CUET Focused Multiple Choice Questions (Part 3)

Q11. Consider the execution of **Insertion Sort** on an array. The number of element shifting operations executed by the inner loop is mathematically equal to which metric of the input array?

(A) The total number of prime numbers in the array
(B) The length of the array multiplied by 2
(C) The total number of inversions present in the original array
(D) The depth of the recursive call stack

Sol. Q11. An inversion is a pair of indices (i, j) such that $i < j$ and $A[i] > A[j]$. In Insertion Sort, every time an element is shifted one position to the right to accommodate the `key`, exactly one inversion is resolved. Hence, the total number of shifting operations executed is strictly equal to the number of inversions in the input array.
Answer: (C)

Q12. Which built-in algorithm is used by Python for its standard `list.sort()` and `sorted()` functions, and what are its best-case and worst-case time complexities?

(A) Quick Sort; Best: $O(n \log n)$, Worst: $O(n^2)$
(B) Merge Sort; Best: $O(n \log n)$, Worst: $O(n \log n)$
(C) Heapsort; Best: $O(n)$, Worst: $O(n \log n)$
(D) Timsort; Best: $O(n)$, Worst: $O(n \log n)$

Sol. Q12. Python utilizes Timsort, a hybrid algorithm combining Merge Sort and Binary Insertion Sort designed by Tim Peters. It detects natural ascending/descending runs, achieving $O(n)$ best-case time on already or partially sorted data, and guarantees $O(n \log n)$ worst-case time with stability.
Answer: (D)

Q13. An array $A = [12, 11, 13, 5, 6]$ is sorted using Insertion Sort. What will be the state of the array after the **second pass** ($i = 2$, processing element 13)?

(A) $[11, 12, 13, 5, 6]$
(B) $[11, 13, 12, 5, 6]$
(C) $[5, 11, 12, 13, 6]$
(D) $[12, 11, 13, 5, 6]$

Sol. Q13. Let us trace Insertion Sort:

- Initial: $[12, 11, 13, 5, 6]$
- Pass 1 ($i = 1$, key $= 11$): 12 shifts right; 11 inserted at index $0 \rightarrow [11, 12, 13, 5, 6]$.
- Pass 2 ($i = 2$, key $= 13$): Compare 13 with 12 ($12 < 13$, loop terminates immediately, 0 shifts). Array remains $[11, 12, 13, 5, 6]$.

Answer: (A)

Q14. In the Lomuto partitioning scheme of Quick Sort with $A = [10, 80, 30, 90, 40, 50, 70]$ where the last element (70) is chosen as the pivot, what is the final index position of the pivot element after completing one full partition step?

(A) Index 3
(B) Index 4
(C) Index 5
(D) Index 2

Sol. Q14. Tracing Lomuto partition with pivot $= 70$:

- Elements smaller than or equal to 70: $10, 30, 40, 50$ (exactly 4 elements).
- Elements strictly greater than 70: $80, 90$ (2 elements).
- During partitioning, the 4 smaller elements are placed into indices $0, 1, 2, 3$.
- Finally, the pivot 70 is swapped with the element at index $i + 1 = 4$.
- Hence, the pivot is placed at index 4, with $[10, 30, 40, 50]$ to its left and $[90, 80]$ to its right.

Answer: (B)

Q15. Which of the following sorting algorithms is most appropriate when sorting a linked list where sequential access is fast ($O(1)$) but random access is slow ($O(n)$), and why?

(A) Quick Sort, because it does not require memory allocation.
(B) Heapsort, because it builds a tree structure.
(C) Merge Sort, because it merges sublists sequentially without requiring random indexing and can be implemented with $O(1)$ extra space on linked lists.
(D) Binary Insertion Sort, because binary search works efficiently on linked lists.

Sol. Q15. Merge Sort is ideally suited for linked lists because merging two sorted linked lists requires only sequential pointer manipulations ($O(1)$ additional space without allocating arrays) and does not rely on random indexing. In contrast, Quick Sort and Heapsort require efficient random access for partitioning and index calculations, making them inefficient on linked lists.
Answer: (C)